

COMP0084 coursework

Anonymous ACL submission

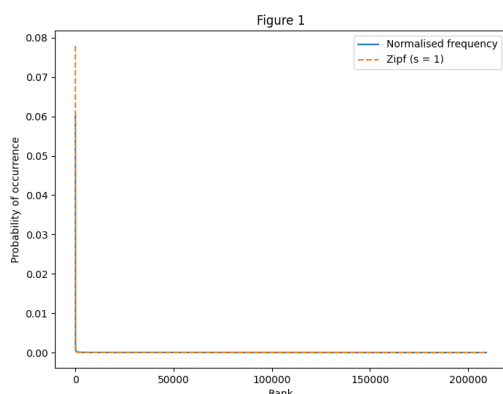


Figure 1: Normalized term frequency vs frequency rank (without stop word removal) compared with Zipf's law at $s=1$

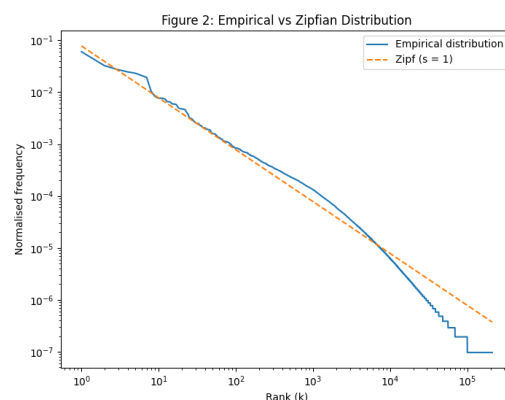


Figure 2: Normalized term frequency vs frequency rank (without stop word removal) compared with Zipf's law at $s=1$ (log-log)

1 Task 1

For text preprocessing, I converted all words to lowercase and removed all non alphanumeric characters from each token. This was to avoid cases where (the/The/the,) were treated as three separate tokens in our investigation, allowing a more natural reflection of the frequency of each term. Finally tokens were obtained by splitting the text at each whitespace

Figure 1 shows that a small number of terms have very high probability of occurrence, while the majority of terms have very low probabilities. The distribution decreases sharply for high-ranked terms and then gradually forms a long tail. This inverse relationship between rank and probability is consistent with Zipf's Law

The empirical distribution follows the general shape of Zipf's law, with a small number of high-frequency terms dominating the corpus and a long tail of rare terms. However, as shown in Figure 2, the empirical curve deviates systematically from the theoretical Zipf distribution defined by Eq. (1) with $s = 1$. The most frequent terms are stop words such as *the*, *of*, *a*, *and*, and *to*, yet even these fall short of what Zipf's law predicts for the top ranks, indicating that the empirical distribution decays more steeply than $s = 1$ assumes. This implies an effective $s > 1$, meaning the canonical Zipf law underestimates the decay rate and therefore overestimates the frequency of mid- and low-ranked terms.

This divergence can be quantified using the product $k \cdot f(k)$, which under the ideal Zipf law should be approximately constant at $C = \frac{1}{H_N} \approx 0.0809$, where $H_N = \sum_{i=1}^N i^{-1}$ for $N = 131,039$. The empirical mean of $k \cdot f(k)$ is only 0.0236, well below this theoretical value, confirming that the corpus follows a steeper power law than $s = 1$ predicts.

Although removing stop words increases the normalised frequency of the remaining terms due to probability mass redistribution, this increase is substantially smaller than what Zipf's law predicts for the highest ranks. After stop-word removal, the most frequent remaining

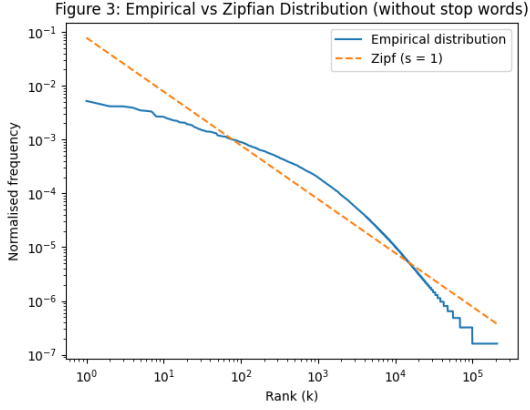


Figure 3: Normalized term frequency vs frequency rank (with stop word removal) compared with Zipf’s law at $s=1$ (log-log)

content words, such as *1*, *2*, *one*, *name*, and *3*, have much lower probabilities than the Zipf model predicts for top-ranked terms, causing a larger underestimation at the head of the distribution. In the mid-rank region, the redistributed probability mass causes the empirical frequencies to exceed the Zipfian prediction, before the tail decays more rapidly than $s = 1$ assumes. This three-region mismatch is more severe than in the case with stop words, as reflected in the mean of $k \cdot f(k)$ increasing from 0.0236 to 0.0384 after stop-word removal, moving closer to the theoretical $C = 0.0809$ on average, yet masking a more pronounced local divergence across all regions of the distribution.

2 Task 2

The inverted index is constructed by iterating over all unique passages, identified by their passage ID (*pid*). Each passage is preprocessed by lowercasing, removing punctuation, and splitting on whitespace, followed by stop-word removal. Stop words are removed at this stage as they dominate the corpus yet carry no discriminative value for relevance ranking, and retaining them would inflate term frequencies in a way that obscures meaningful content terms. For each token, the index maps the term to a dictionary of passage IDs and their corresponding term frequencies, i.e. $term \rightarrow \{pid : count\}$. Storing term frequency per passage is sufficient for the relevance ranking models employed, such as TF-IDF and BM25,

while storing additional information such as term positions would introduce unnecessary memory overhead with negligible benefit for these models.

3 Task 4

Dirichlet smoothing is expected to perform best. Unlike Laplace and Lidstone, Dirichlet smoothing adapts the degree of smoothing to passage length, applying stronger smoothing to shorter passages where term frequency estimates are less reliable. This is particularly well-suited to passage retrieval, where passage lengths vary considerably. This is supported empirically by the query *“What are the social determinants of health?”*, where even the bottom-ranked out of the 100 passages retrieved by Dirichlet remained topically relevant, discussing social, economic, and behavioural health factors. In contrast, the bottom-ranked passages under Laplace and Lidstone included documents about gerontological social worker salaries and health and social care qualifications, which are tangentially related at best.

Laplace and Lidstone smoothing are expected to produce the most similar results, as both are additive smoothing methods that add a fixed constant to all term counts regardless of passage length or corpus statistics. Lidstone generalises Laplace by using $\epsilon = 0.1$ rather than adding 1, producing a less aggressive smoothing, but the underlying mechanism is identical. Empirically comparing the same query as above, both Laplace and Lidstone had the same 10 documents ranked in the top 10. With 6 of the 10 being ranked in the exact same positions.

$\epsilon = 0.1$ is a reasonable starting point as it smooths less aggressively than Laplace ($\epsilon = 1$), preserving more of the observed term frequency signal. However, it remains a fixed global value with no sensitivity to passage length or collection statistics. As we saw from our chosen query, ϵ values at and above 0.1 begin to increasingly resemble Laplace, offering little practical advantage over using Laplace directly. A better range would likely be $\epsilon \in [0.01, 0.1]$ to allow observed term frequency signal more of a chance to influence ranking.

Increasing μ from 50 to 5000 would likely be

too aggressive for this setting. The Dirichlet parameter μ controls the balance between the passage-level term frequency and the corpus-level language model; larger μ places more weight on the corpus distribution and less on the passage itself. At $\mu = 50$, the smoothing is relatively light, trusting the passage’s own term frequencies more. At $\mu = 5000$, the corpus prior would dominate for most passages, effectively flattening the passage-level signal.

4 Task 5

4.1 BM25 Optimization

Average Precision measures ranking quality for a single query. It rewards systems that place relevant documents higher in the ranked list. For a ranked list of binary relevance judgements:

$$AP = (1/R) * \sum_{k=1}^n [P(k) * rel(k)] \quad (1)$$

where R = number of relevant documents, $P(k)$ = precision at rank k , $rel(k) = 1$ if document at rank k is relevant, else 0. Mean Average Precision (MAP) is the macro-average of AP across all queries.

NDCG@ k measures ranking quality with a logarithmic discount, cut off at position k . It is normalised against the ideal ranking (IDCG):

$$DCG@k = \sum_{i=1}^k \frac{rel(i)}{\log_2(i+1)} \quad (2)$$

$$NDCG@k = \frac{DCG@k}{IDCG@k} \quad (3)$$

The logarithmic discount means a relevant document at rank 1 contributes far more than one at rank 10, making NDCG sensitive to the top of the ranked list. I use $k=10$ throughout. For evaluating models I use the average NDCG over all queries

The inverted index is generated over the sampled train dataset and is used throughout the pipeline to generate tf and idf dictionaries along with all other data structures needed to calculate bm25. It is generated using the same preprocessing steps as the previous index generated in task 2.

4.1.1 Dataset Note

Due to the sheer size of the dataset, I have chosen to sample by query and all the following data is retrieved by sampling from $N = 1500$ queries from the training dataset. The full validation set is used. I note that the distribution of relevant to non-relevant documents per query is heavily skewed; however, this reflects real deployment scenarios and I therefore opted for this sampling strategy over class-weight redistribution.

4.1.2 Hyperparam Optimization

The BM25 free parameters k_1 and b control term-frequency saturation and document-length normalisation respectively. k_2 , which governs term-frequency weighting in the query, was held constant at $k_2 = 100$ throughout. Because query terms in short keyword queries rarely repeat, k_2 has negligible practical effect on ranking and fixing it reduces the search space without sacrificing expressiveness.

The parameters k_1 and b were tuned via exhaustive grid search over the training sample. The selection criterion was the sum $MAP + (Mean)NDCG@10$, chosen to balance overall retrieval quality (MAP) with top-of-ranking precision (NDCG@10). The search grid was:

$$k_1 \in \{1.2, 1.5, 1.75\}, \quad b \in \{0.5, 0.75, 0.8\}$$

For each parameter configuration, all documents in the index were scored and ranked by descending BM25 score for every training query. The ranked list was then merged with ground-truth relevance labels to produce a ranked relevance sequence, from which AP and NDCG@10 were computed per query. The two metrics were then macro-averaged across all training queries to produce the MAP and (Mean)NDCG@10 figures used as the selection criterion. Validation statistics were computed using the same configuration simply substituting the training set for the validation set.

The optimal configuration was $k_1 = 1.2$, $b = 0.5$.

The sensitivity analysis revealed an asymmetry worth noting: varying k_1 produced more pronounced changes in the objective than varying b , suggesting that term-frequency

saturation is the dominant factor for this corpus. A lower $k_1 = 1.2$ means BM25 saturates term-frequency contributions more aggressively — preventing highly repeated query terms from dominating the score — which is appropriate for a passage-retrieval setting where documents tend to be short and topically focused. The relatively low $b = 0.5$ (partial length normalisation) similarly makes sense here: aggressive length normalisation ($b \rightarrow 1$) would penalise longer passages that may legitimately contain more relevant content.

4.1.3 Evaluation Results

k1	b	MAP	(Mean)NDCG@10	MAP+(Mean)NDCG
1.20	0.50	0.223618	0.269573	0.493191
1.20	0.75	0.222991	0.269726	0.492717
1.20	0.80	0.221566	0.267332	0.488898
1.50	0.80	0.214082	0.256704	0.470786
1.50	0.75	0.213633	0.256530	0.470163
1.50	0.50	0.213577	0.256343	0.469920
1.75	0.75	0.207471	0.249963	0.457434
1.75	0.80	0.207611	0.249312	0.456922
1.75	0.50	0.204920	0.247241	0.452161

Table 1: BM25 parameter search results on the training set (sampled). Best parameters: $k_1 = 1.2$, $b = 0.5$.

Split	MAP	(Mean)NDCG@10
Training (sampled)	0.223618	0.269573
Validation (full)	0.069700	0.079100

Table 2: BM25 performance with best parameters ($k_1 = 1.2$, $b = 0.5$) across splits.

The validation MAP is substantially lower than the training figure, and several compounding factors explain this gap.

First, corpus incompleteness: the inverted index and BM25 score calculations are derived from the sampled training corpus. On unseen validation queries, relevant documents may reference vocabulary or passages not present in the index, making the retrieval task materially harder, the model is operating off an incomplete picture of the document space, and this is far more apparent on out-of-distribution queries.

Second, label sparsity: the relevant-document distribution is heavily right-skewed. Many queries have only one or two judged-relevant documents, meaning a single missed retrieval has an outsized impact on AP for

that query. When averaged across queries, this sparsity structurally depresses MAP regardless of ranking quality.

Third, train-validation distributional shift: hyperparameters were selected to maximize performance on the sampled training queries. It is plausible that those 1500 queries are not perfectly representative of the full validation distribution, introducing a mild overfitting effect at the level of hyperparameter selection.

Together, these factors suggest the validation MAP should be interpreted as a lower bound on what BM25 could achieve with a complete corpus and denser relevance judgments, rather than as a true reflection of the ranker’s quality. The NDCG@10 gap is typically smaller because it focuses on the top of the ranking, where BM25’s lexical matching is most reliable and corpus incompleteness is less damaging.

4.2 Logistic Regression Implementation

The logistic regression model was implemented from scratch using NumPy. The model uses the standard sigmoid activation to produce probabilities:

$$\hat{y} = \sigma(Xw + b) = \frac{1}{1 + e^{-z}} \quad (4)$$

Training used mini-batch gradient descent, the training set was shuffled each epoch and split into fixed-size batches. For each batch, the gradient of the binary cross-entropy loss was computed and parameters updated:

$$w \leftarrow w - \frac{\alpha}{|B|} X_B^T (\hat{y}_B - y_B), \quad b \leftarrow b - \frac{\alpha}{|B|} \sum (\hat{y}_B - y_B) \quad (5)$$

No regularization or early stopping was applied; all models ran for a fixed 100 epochs.

4.2.1 Dataset Setup

Due to the sheer size of the dataset and limited compute resources, I have chosen to sample by query and all the following data is retrieved by sampling from $N = 500$ queries from the training dataset. The full validation set is used. I note that the distribution of relevant to non-relevant documents per query is heavily skewed; however, this reflects real deployment scenarios and I therefore opted for

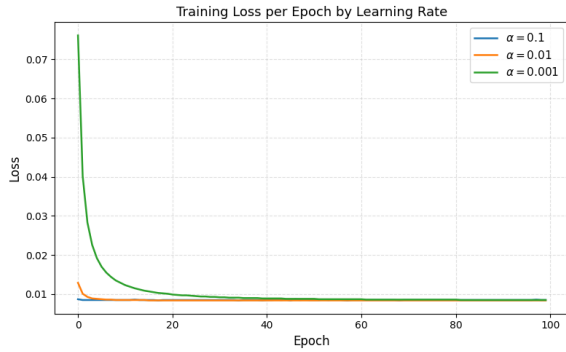


Figure 4: Model Loss vs epoch for learning rates [0.1,0.01,0.001]

this sampling strategy over class-weight redistribution.

GloVe (Wikipedia + Gigaword, 100-dimensional) embeddings were loaded via ‘gensim’. Each query and passage was represented as the mean of its token embeddings (tokens not in the vocabulary were skipped). The final feature vector for each query–passage pair was the concatenation of both mean vectors $\rightarrow$ 200-dimensional input. Binary relevance labels were used from ‘training-data.tsv’ and ‘validation-data.tsv’. A ‘StandardScaler’ was fit on the training features only, then applied to both train and validation sets to zero-centre and unit-variance the inputs, important for stable gradient descent.

Three models were trained independently across learning rates [0.1, 0.01, 0.001], sharing the same mini-batch schedule and epoch budget. The loss curve across epochs was recorded for each model.

4.2.2 Analysis

Learning Rate	MAP	(Mean)NDCG@10
0.1	0.0184	0.0158
0.01	0.0181	0.0152
0.001	0.0179	0.0144

Table 3: Neural model performance across learning rates. All configurations perform poorly, suggesting issues beyond hyperparameter tuning.

The three learning rates produce meaningfully different convergence behaviours, though all ultimately arrive at the same terminal loss of approximately 0.0084–0.0085.

At $\alpha = 0.1$, the model begins at a loss of 0.0087 and reaches its floor within the first

epoch, remaining essentially flat for the entire 100-epoch run. The loss fluctuates slightly (between 0.0084 and 0.0087) but shows no meaningful downward trend, indicating that the model overshoots any useful gradient signal immediately and spends the remainder of training oscillating around a fixed point.

At $\alpha = 0.01$, the model starts higher at 0.0129 and descends steadily over the first 15 epochs before plateauing at 0.0084 for the remaining 85 epochs. This is the most textbook convergence curve of the three — a clean, monotonic descent followed by a stable floor — suggesting the learning rate is well-calibrated to the loss landscape.

At $\alpha = 0.001$, the model begins at a substantially elevated loss of 0.0761 and descends gradually but continuously across all 100 epochs, only approaching the shared floor near epoch 80. The descent never fully flattens within the training window, suggesting that with sufficient additional epochs it would converge to the same point.

The fact that all three learning rates converge to the same loss floor is highly revealing. In a well-posed learning problem, a higher learning rate risks overshooting a true minimum, yielding a worse terminal loss. Here, the identical terminal values across $\alpha \in \{0.1, 0.01, 0.001\}$ indicate that the optimizer is not descending toward a meaningful minimum at all — it is converging to a trivial solution accessible from almost any initialization.

This is consistent with the two structural failure modes of this model. The severe class imbalance (only 1-2 relevant documents per query) means the model can minimize loss to near-zero simply by predicting non-relevant for every input, with no need to learn any ranking signal. This is compounded by the use of averaged word embeddings as input representations: collapsing a passage into a single mean vector discards all keyword, positional, and frequency information, leaving the model with inputs that carry almost no discriminative signal between relevant and non-relevant documents.

Together, these factors produce a model that converges successfully on a degenerate solution. The low training loss is therefore not evidence of good learning but of the opposite, the model has found the easiest possible

path through the loss landscape. This directly explains the near-zero MAP and NDCG@10 scores observed at validation: a model that has learned only to predict the majority class produces rankings that are effectively random with respect to true relevance.